# OOPS CONCEPTS

## ★ What is OOPS ?

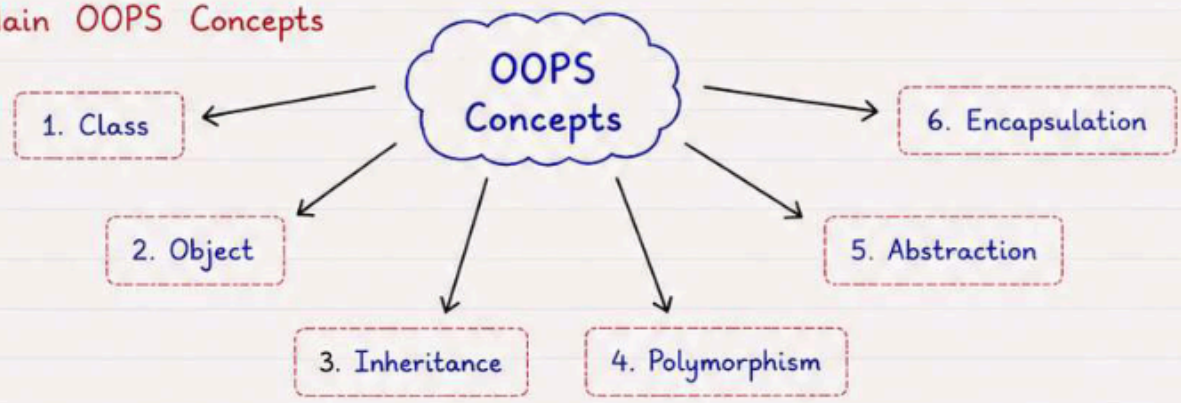
OOPS (Object-Oriented Programming System) is a programming paradigm that uses "Objects" and "Classes" to design and structure software.

It combines data (attributes) and code (methods) into a single unit called an object.

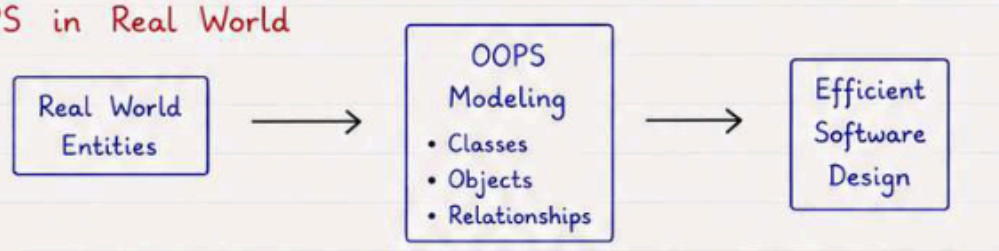
## ★ Why OOPS ?

- It makes programs easier to understand and maintain.
- It provides data security through encapsulation.
- It supports code reusability through inheritance.
- It allows flexibility through polymorphism.
- It helps in modeling real-world entities.

## ★ Main OOPS Concepts




## ★ OOPS in Real World




We represent real world entities as objects in our programs.

## ★ Summary

OOPS helps to break down a complex problem into smaller, manageable parts using objects and classes.

It improves code organization, reusability and scalability.

These concepts form the foundation of strong software design.

**Remember:**

Think in Objects,
Design with Classes,
Code with Purpose! ☆

# OOPS CONCEPT FULL HANDBOOK

## 1. CLASS

### ★ What is a Class ?

A class is a **blueprint or template** for creating objects.
It defines a set of **attributes (data)** and **methods (functions)** that the objects of the class will have.

### ★ Key Points

- A class does **not** occupy memory.
- It is a **user-defined data type**.
- It acts as a **blueprint** for creating objects.
- It helps in **data hiding** and **code reusability**.

### ★ Syntax (in Java)

```
class ClassName {
        // Attributes
        dataType attribute1;
        dataType attribute2;
        ...
        // Methods
        returnType methodName(parameters) {
            // method body
        }
}
```

→ Class Keyword

→ Attributes / Variables (Data Members)

→ Methods / Functions (Member Methods)

### ★ Example

```
class Student {
      // Attributes
      int id;
      String name;
      double marks;
      // Methods
      void display() {
         System.out.println("ID: " + id);
         System.out.println("Name: " + name);
         System.out.println("Marks: " + marks);
      }
}
```

**Class Structure**

| Student (Class Name) | |
|---|---|
| Attributes (Data)<br>int id;<br>String name;<br>double marks; | → Holds Data (State) |
| Methods (Functions)<br>void display(); | → Performs Actions (Behavior) |



**Remember :**
A class is just a **template**. No memory is allocated when a class is created. Memory is allocated **only when objects are created** from the class.

**Example :**

```
Student s1 = new Student();
// Object creation
```

# OOPS CONCEPT FULL HANDBOOK

## 2. OBJECT

### ★ What is an Object ?

An object is a real-world entity which is an instance of a class.
It has state (data) and behavior (methods).

### ★ Key Points

- An object is created from a class.
- It occupies memory.
- Each object has its own copy of data.
- Objects can interact by calling methods.
- Multiple objects can be created from the same class.

### ★ Syntax (in Java)

```
ClassName objectName = new ClassName();
```

- → Class Name
- → Object Name (Reference)
- → new Keyword
- → Constructor

### ★ Example

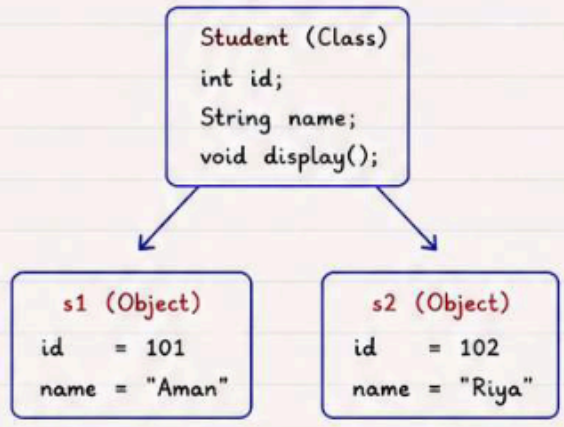
```
class Student {
    int id;
    String name;

    void display() {
        System.out.println(id + " - " + name);
    }
}
public class Main {
    public static void main(String[] args) {
        // Creating objects
        Student s1 = new Student();
        Student s2 = new Student();

        // Setting values
        s1.id = 101;
        s1.name = "Aman";
        s2.id = 102;
        s2.name = "Riya";

        // Calling method
        s1.display();    // Output: 101 - Aman
        s2.display();    // Output: 102 - Riya
    }
}
```

#### Object Diagram




Here, s1 and s2 are objects (instances) of class Student.
Each object has its own data.



**Remember :**

- Class is a blueprint.
- Object is a real entity created from the class.
- Many objects can be created from one class.

**Example in Real Life :**

Class : Mobile
Objects : iphone, samsung, oneplus (each mobile has its own details and behavior)

# OOPS CONCEPT FULL HANDBOOK

## 3. METHOD

### ★ What is a Method ?

A method is a block of code that performs a specific task.
It is defined inside a class and is used to define the behavior of objects.

### ★ Key Points

- A method is a function that belongs to a class.
- It is used to perform some operation.
- It can access and modify the data (attributes) of the class.
- Methods improve code modularity and reusability.
- Methods can return a value or may not return anything (void).

### ★ Syntax (in Java)

```
returnType methodName(parameters) {
        // method body
        // statements
        return value;   // optional
}
```

- `returnType` → Return type of method (int, void, String, etc.)
- `methodName` → Name of the method
- `parameters` → Parameters passed to the method (optional)
- `method body` → Method body

### ★ Example

```
class Calculator {
    int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
    void showMessage() {
        System.out.println("Hello, Java!");
    }
}
public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        int result = c.add(10, 20);   // calling method with return
        System.out.println("Sum = " + result);
        c.showMessage();              // calling void method
    }
}
```

#### How Method Works ?




Object calls the method.
Method performs the task.
Method returns the result (if any).

**Remember :**

- Method is the behavior of an object.
- It is always defined inside a class.
- A method can return a value or nothing.
- Good method names describe what the method does.

**Example in Real Life :**

Think of a remote control.
Pressing a button (method) performs an action like changing channel, increasing volume, etc.
Each button has a specific function.

# OOPS CONCEPT FULL HANDBOOK

## 4. CONSTRUCTOR

### ★ What is a Constructor ?

A constructor is a special method in a class that is used to initialize objects. It is called automatically when an object is created.

### ★ Key Points

- Constructor name is same as class name.
- It has no return type, not even void.
- It is called automatically when object is created.
- Used to initialize object's data.
- A class can have multiple constructors (constructor overloading).
- If no constructor is written, Java provides a default constructor.

### ★ Syntax (in Java)

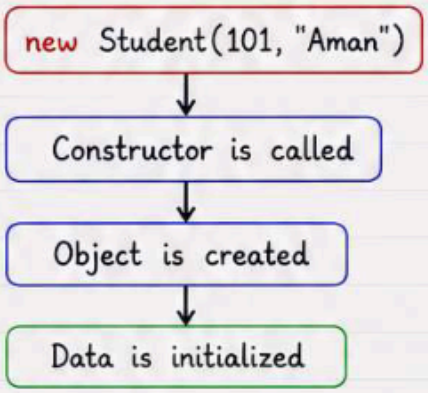
```
class ClassName {
      ClassName(parameters) {
         // initialization code
      }
}
```

→ Constructor name
→ Parameters (optional)
→ Constructor body

### ★ Example

```
class Student {
      int id;
      String name;
      // Constructor
      Student(int id, String name) {
          this.id = id;
          this.name = name;
      }
      void display() {
          System.out.println(id + " - " + name);
      }
}
public class Main {
   public static void main(String[] args) {
       Student s1 = new Student(101, "Aman");
       Student s2 = new Student(102, "Riya");
       s1.display();   // Output: 101 - Aman
       s2.display();   // Output: 102 - Riya
   }
}
```

### How Constructor Works ?




### Types of Constructors

- Default Constructor (no-arg)
- Parameterized Constructor
- Copy Constructor (via parameter)

### Remember :

- Constructor runs only once when object is created.
- It helps to initialize object in a clean way.
- Constructors have no return type.

### Example in Real Life :

When you register in any website, your details (name, email, password) are filled automatically in the form. Constructor works the same way.

# OOPS CONCEPT FULL HANDBOOK

## 5. INHERITANCE

### ★ What is Inheritance ?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent/superclass). It promotes code reusability and establishes an IS-A relationship.

### ★ Key Points

- The class whose members are inherited is called superclass (parent class).
- The class that inherits is called subclass (child class).
- The subclass can have its own members in addition to inherited members.
- Inheritance supports code reusability and extensibility.
- Java does not support multiple inheritance with classes (but supports it using interfaces).

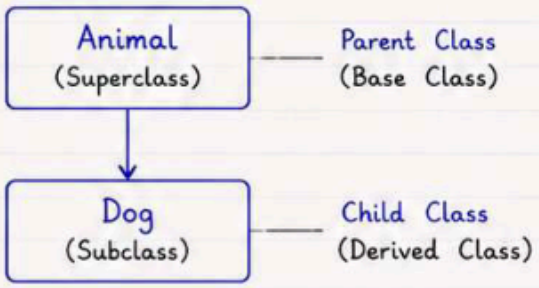
### ★ Syntax (in Java)

```
class SuperClass {                          → Parent / Superclass
    // fields and methods
}
class SubClass extends SuperClass {         → Child / Subclass
    // own fields and methods
}
```

### ★ Example (in Java)

```
// Parent Class (Superclass)
class Animal {
    String color = "Brown";
    void eat() {
        System.out.println("Animal is eating");
    }
}
// Child Class (Subclass)
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
// Main Class
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();       // inherited method
        d.bark();      // own method
        System.out.println(d.color); // inherited field
    }
}
```

### Inheritance Diagram




### How it Works ?

1. Dog is a Animal. (IS-A Relationship)
2. Dog inherits color and eat() from Animal.
3. Dog can use inherited members and also define its own members.

| Benefits | Remember |
|---|---|
| • Code Reusability | ✓ Use extends keyword to inherit. |
| • Easy Maintenance | ✓ Private members are not inherited. |
| • Extensibility | ✓ Constructors are not inherited. |
| • Better Organization | ✓ Inheritance represents IS-A relationship. |

# OOPS CONCEPT FULL HANDBOOK

## 6. POLYMORPHISM

### ★ What is Polymorphism ?

Polymorphism means "many forms". It allows one interface to be used for different types.

In Java, polymorphism is achieved by Method Overloading and Method Overriding.

### ★ Types of Polymorphism

1. Compile-time Polymorphism (Method Overloading)
2. Run-time Polymorphism (Method Overriding)

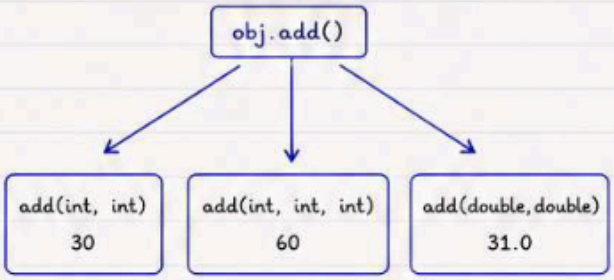
### 1. Compile-time Polymorphism (Method Overloading)

- Occurs in the same class.
- Same method name but different parameter list (number, type or order).
- It is resolved at compile time.

Example (in Java)

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
    double add(double a, double b) {
        return a + b;
    }
}
public class Main {
   public static void main(String[] args) {
      Calculator obj = new Calculator();
      System.out.println(obj.add(10, 20));       // 30
      System.out.println(obj.add(10, 20, 30));   // 60
      System.out.println(obj.add(10.5, 20.5));   // 31.0
   }
}
```

How it Works ?




The compiler decides which method to call based on the arguments passed.

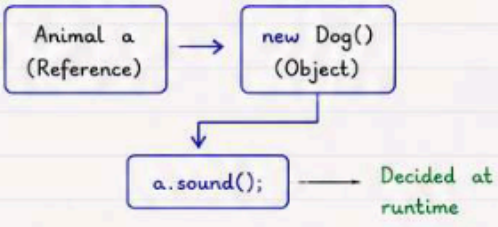
### 2. Run-time Polymorphism (Method Overriding)

- Occurs in different classes using inheritance.
- Same method name, same parameters.
- It is resolved at runtime.

Example (in Java)

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
   public static void main(String[] args) {
      Animal a = new Dog();   // Reference of parent, object of child
      a.sound();              // Calls overridden method
   }
}
```

How it Works ?




JVM decides at runtime which method to call based on the actual object.

### Key Difference

| Feature | Method Overloading | Method Overriding |
|---|---|---|
| Type | Compile-time | Run-time |
| Where | Same class | Different class (Inheritance) |
| Parameters | Different parameter list | Same parameters |
| Binding | Static binding | Dynamic binding |

### Remember

- Overloading increases readability.
- Overriding achieves runtime polymorphism.
- @Override annotation is good practice.
- Polymorphism improves code reusability and flexibility.

# OOPS CONCEPT FULL HANDBOOK

# 7. ABSTRACTION & INTERFACE

## 1. Abstraction

★ Abstraction is the process of hiding implementation details and showing only the essential features to the user.

★ It helps to reduce complexity and improve security.

★ In Java, abstraction is achieved using abstract class and interface.

**Purpose of Abstraction**

- Hide internal details
- Show only functionality
- Reduce complexity
- Improve maintainability

## 2. Abstract Class

- A class that cannot be instantiated.
- Can have abstract methods (without body) and concrete methods (with body).
- Child class must extend and implement abstract methods.

**Example (in Java)**

```
abstract class Shape {
  abstract void draw();   // abstract method

  void color() {
    System.out.println("Color is set"); // concrete method
  }
}
class Circle extends Shape {
  void draw() {
     System.out.println("Drawing Circle");
  }
}
```

**How it Works ?**

Shape (abstract class) abstract void draw();
↓
Circle (Child class) implements draw()
↓
Object of Circle draw() is executed

## 3. Interface

- An interface is a blueprint of a class.
- It contains only abstract methods (by default public and abstract) and constants.
- A class implements an interface using the implements keyword.
- Multiple interfaces can be implemented.

**Example (in Java)**

```
interface Animal {
    void sound();      // public & abstract
}
class Dog implements Animal {
  public void sound() {
     System.out.println("Dog barks");
  }
}
class Main {
 public static void main(String[] args) {
   Animal a = new Dog();
   a.sound();
}
```

**How it Works ?**

Animal (Interface) void sound();
↓
Dog (Class) implements sound()
↓
Animal a = new Dog(); a.sound();
↓
Output: Dog barks

## 4. Key Points

- Abstract class can have constructor, non-abstract methods and variables.
- Interface cannot have constructor.
- From Java 8, interface can have default and static methods.
- From Java 9, interface can have private methods.

**Abstract Class**

- Uses 'extends' keyword
- Can have abstract + concrete methods
- Can have variables
- Single inheritance
- Can have constructor

**Interface**

- Uses 'implements' keyword
- Only abstract methods (by default)
- Only constants
- Multiple inheritance
- Cannot have constructor

## 5. Abstract Class vs Interface

| Feature | Abstract Class | Interface |
|---|---|---|
| Definition | A class with abstract and concrete methods. | A blueprint with only abstract methods (by default). |
| Keyword | abstract class | interface |
| Inheritance | Single (only one abstract class can be extended) | Multiple (multiple interfaces can be implemented) |
| Methods | Abstract + Concrete methods | Only abstract methods (default public & abstract) |
| Variables | Can have variables | Only constants (public static final) |
| Constructor | Can have constructor | Cannot have constructor |
| Access Modifier | Can be any | Methods are public by default |
| Use | When we want common behavior + state | When we want common behavior (no state) |

**Pro Tip:** Use abstract class when you want to share code. Use interface when you want to define a contract for behavior.

# OOPS CONCEPT FULL HANDBOOK

# 8. ENCAPSULATION

## ★ What is Encapsulation ?

Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of the object's components. It helps in data hiding and protects the data from unauthorized access.

## ★ Key Points

- We achieve encapsulation by using access modifiers.
- Class variables should be private.
- Access to variables is provided through public getter and setter methods.
- It protects data from being modified by accident.
- It improves code maintainability and flexibility.

## ★ Syntax (in Java)

```
class ClassName {
    private dataType variable;    // private variable

    // getter method
    public dataType getVariable() {
        return variable;
    }

    // setter method
    public void setVariable(dataType value) {
        variable = value;
    }
}
```

## ★ Example (in Java)

```
class Student {
    private int id;
    private String name;

    // Getter for id
    public int getId() {
        return id;
    }

    // Setter for id
    public void setId(int id) {
        this.id = id;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }
}
```

## ★ How it Works ?

**Client Code**

```
obj.setId(101);
int id = obj.getId();
```

→

**Student Class**

Private Data
- id
- name

Public Methods
+ setId()
+ getId()
+ setName()
+ getName()

## Benefits

- ☑ Protects data from unauthorized access.
- ☑ Improves security.
- ☑ Allows controlled access to data.
- ☑ Makes code easy to maintain and modify.

## Remember

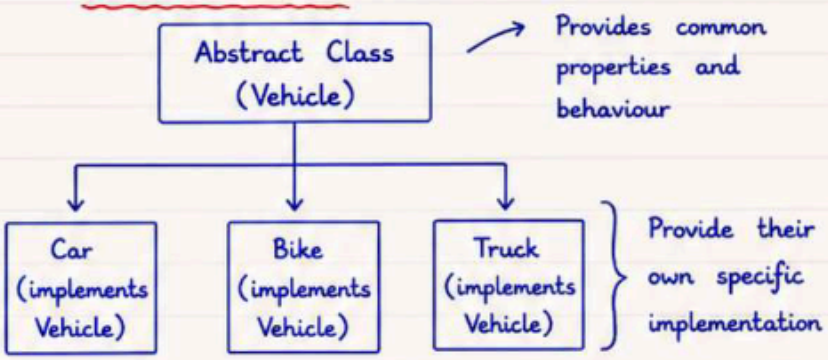
- ☑ Make variables private.
- ☑ Provide public getter and setter methods.
- ☑ Access and modify data only through methods.

# Java Abstraction and Interface

## 1. ABSTRACT CLASS - DEEP DIVE

An abstract class is a class that cannot be instantiated and is meant to be inherited by other classes.

### How it works ?




## 2. KEY FEATURES OF ABSTRACT CLASS

- It can have abstract and non-abstract methods.
- It can have constructors.
- It can have instance variables (state).
- It can have static methods.
- It cannot be instantiated.
- It is extended using "extends" keyword.
- A subclass must provide implementation for all abstract methods.

**WHAT IS ALLOWED IN ABSTRACT CLASS ?**

| Feature | Allowed ? |
|---|---|
| Abstract Methods | ✓ |
| Concrete Methods | ✓ |
| Constructors | ✓ |
| Instance Variables | ✓ |
| Static Methods | ✓ |
| Final Methods | ✓ |
| Object Creation | ✗ |

## 3. EXAMPLE IN JAVA

```
abstract class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    abstract void start();   // abstract method

    void stop() {
        System.out.println("Vehicle Stopped");
    }
}
```

→ Vehicle is an abstract class.

→ It has an abstract method start() without implementation.

→ It has a concrete method stop() with implementation.

→ Constructor is used to initialize state (brand).

## 4. IMPORTANT NOTES

- We cannot create object of an abstract class.
- If a class extends an abstract class, it must implement all abstract methods or itself be declared abstract.
- Abstract class helps in providing a common base and partial implementation.

# Java Abstraction and Interface

## 1. INTERFACE - DEEP DIVE

An interface is a blueprint of a class. It defines a contract that classes must follow.

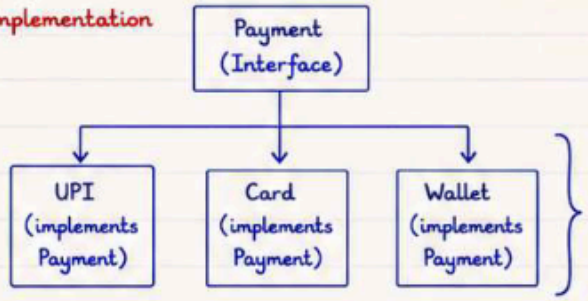
### Why Interface ?

- Achieve 100% abstraction.
- Multiple inheritance is possible.
- Loose coupling and flexibility.
- Used for defining a standard behavior.

## 2. INTERFACE STRUCTURE

```
interface Payment {
    void pay();      // abstract method
    void refund();   // abstract method
}
```

Implementation




Multiple classes can implement the same interface.

## 3. TYPES OF METHODS IN INTERFACE (Modern Java)

| Type | Syntax | Can have body ? | From Version | Example |
|---|---|---|---|---|
| Abstract Method | `void show();` | No (by default) | All Versions | `void show();` |
| Default Method | `default void show() { ... }` | Yes | Java 8 | `default void show() { System.out.println("Hi"); }` |
| Static Method | `static void show() { ... }` | Yes | Java 8 | `static void show() { System.out.println("Hi"); }` |
| Private Method | `private void show() { ... }` | Yes (Only inside interface) | Java 9 | `private void show() { // helper method }` |

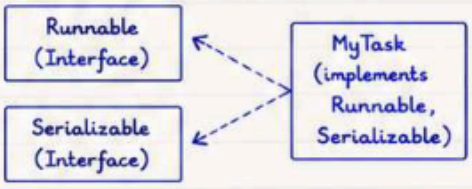
## 4. CONSTANTS IN INTERFACE

- All variables in interface are public static final by default.
- They must be initialized.

```
interface Config {
    int MAX_LIMIT = 100;    // public static final
    String APP_NAME = "MyApp";
}
```

## 5. MULTIPLE INHERITANCE

- A class can implement multiple interfaces.
- This is not possible with classes.




## 6. EXAMPLE - INTERFACE IN REAL WORLD

```
interface Notification {
    void send(String msg);
    default void printHeader() {
        System.out.println("--- Notification ---");
    }
}
```

Implementation →

```
class EmailNotification implements Notification {
    public void send(String msg) {
        System.out.println("Email : " + msg);
    }
}

class SMSNotification implements Notification {
    public void send(String msg) {
        System.out.println("SMS : " + msg);
    }
}
```

Use →

Client uses Notification interface

- ✓ Loose coupling
- ✓ Easy to extend
- ✓ Follows contract

@aman_views

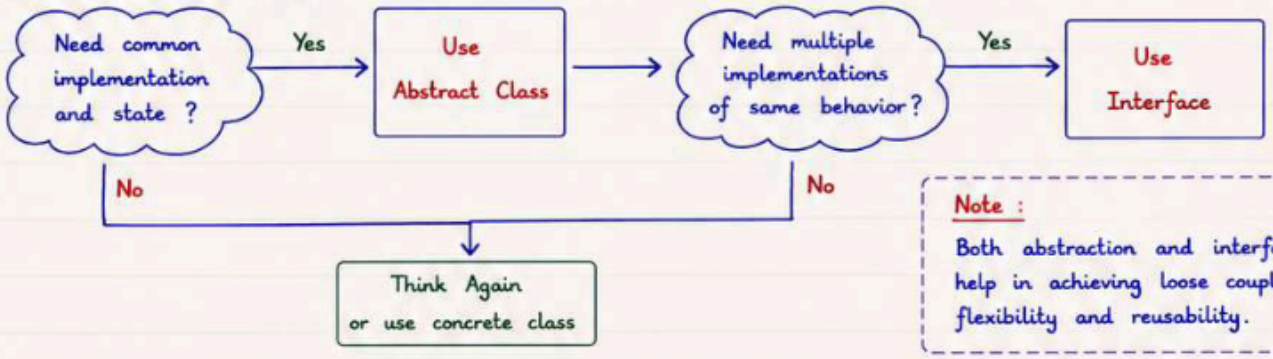
# Java Abstraction and Interface

## 4. ABSTRACT CLASS vs INTERFACE

Complete Comparison

| Feature | Abstract Class | Interface |
|---|---|---|
| 1. Declaration Keyword | abstract keyword | interface keyword |
| 2. Methods | Can have abstract and concrete methods. | Can have only abstract methods (till Java 7). From Java 8 → all types. |
| 3. Method Body | Abstract methods have no body. Concrete methods have body. | Abstract methods have no body. (default/static methods can have body) |
| 4. Default Methods | ✗ | Yes (Java 8 and above) |
| 5. Static Methods | Can have static methods. | Yes (Java 8 and above) |
| 6. Private Methods | Can have private methods. | Yes (Java 9 and above) |
| 7. Variables (Instance) | Can have instance variables. | Cannot have instance variables. |
| 8. Variables (Static) | Can have static variables. | Can have static final variables (acts as constant). |
| 9. Constructors | Can have constructors. | Cannot have constructors. |
| 10. Multiple Inheritance | No (A class can extend only one class.) | Yes (A class can implement multiple interfaces.) |
| 11. Object Creation | Cannot create object. | Cannot create object. |
| 12. Access Modifiers | Can use any access modifier. | By default → public (methods are public) |
| 13. State (Data) | Can maintain state. | Cannot maintain state. |
| 14. Extends / Implements | Use extends keyword. | Use implements keyword. |
| 15. Performance | Better (slightly faster) | Slightly lower (negligible) |
| 16. Use Case | To provide common implementation and state. | To define a contract (behavior). |
| 17. When to use ? | When you want to provide common code and maintain state. | When you want to define behavior that multiple classes must follow. |
| 18. Example | `abstract class Vehicle { abstract void start(); void stop() { ... } }` | `interface Payment { void pay(); default void receipt() { ... } }` |

### Quick Decision Guide




**Note :**
Both abstraction and interface help in achieving loose coupling, flexibility and reusability.

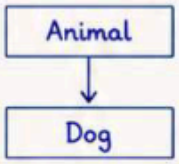
@aman_views

# Java Abstraction and Interface

## 5. IS-A vs HAS-A RELATIONSHIP

### 1. IS-A Relationship (Inheritance)

In inheritance (IS-A), one class is a type of another class. It represents a parent-child relationship.



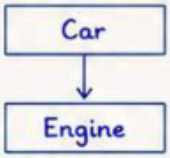

Dog IS-A Animal.

Java Example :

```
class Animal { }
class Dog extends Animal { }
```

### 2. HAS-A Relationship (Composition)

In composition (HAS-A), one class contains an object of another class. It represents an owner-component relationship.




Car HAS-A Engine.

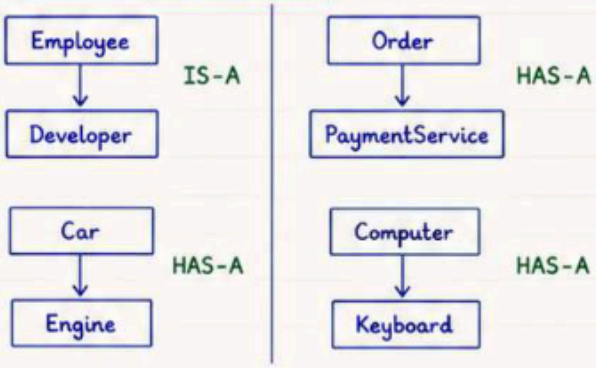
Java Example :

```
class Engine { }
class Car {
    Engine engine = new Engine();
}
```

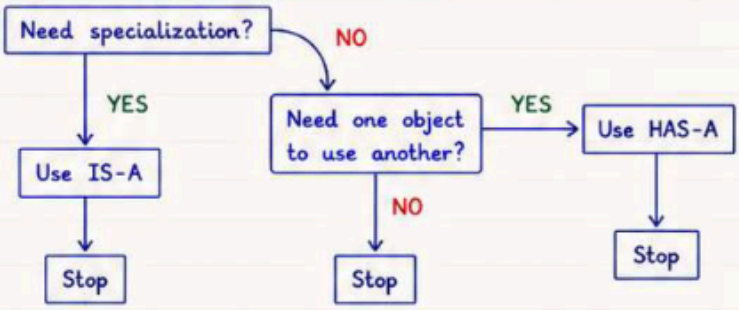
### 3. IS-A vs HAS-A (Quick Comparison)

| Aspect | IS-A (Inheritance) | HAS-A (Composition) |
|---|---|---|
| Meaning | One class is a type of another | One class contains another object |
| Keyword | extends | Object reference |
| Coupling | Tight Coupling | Loose Coupling |
| Reuse | Reuse through Inheritance | Reuse through Composition |
| Relationship | Parent-Child | Owner-Component |
| Flexibility | Less Flexible | More Flexible |
| Best Use Case | When there is a true parent-child relationship | When one class uses another class for its functionality |

### 4. Real Project Examples




### 5. When to Use? (Decision Flow)




### 6. Best Practice

- Prefer Composition (HAS-A) over Inheritance (IS-A) when possible.
- Use IS-A only when there is a true parent-child relationship.
- Use HAS-A for flexibility and loose coupling.

@aman_views

# Java Abstraction and Interface

## 6. JAVA 8 INTERFACE FEATURES & FUNCTIONAL INTERFACES

### 1. Why Java 8 changed Interfaces ?

- Earlier interfaces had only abstract methods.
- Java 8 added Default and Static methods.
- This make interfaces more powerful and suitable for real world use.

### 2. New Features in Interface (Java 8+)

1. Default Methods → has a body, provides default implementation.
2. Static Methods → belongs to interface, not to object.
3. Private Methods → helper methods inside interface (Java 9+).
4. Multiple Inheritance → a class can implement multiple interfaces.

### 3. Default Method Example

```
interface Vehicle {
    void start();                // abstract method
    default void stop() {        // default method
        System.out.println("Vehicle Stopped");
    }
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car Started");
    }
}
```

Child class is not forced to override default method.

### 4. Static Method in Interface

```
interface MathUtil {
    static int add(int a, int b) {
        return a + b;
    }
}

// Usage
int res = MathUtil.add(10, 20);
```

Called using Interface name. Not through object.

### 5. Private Method in Interface (Java 9+)

```
interface MessageService {
    default void send(String msg) {
        if (isValid(msg)) {
            System.out.println("Message Sent");
        }
    }
    private boolean isValid(String msg) {
        return msg != null && !msg.isEmpty();
    }
}
```

Private helper method used inside interface only.

### 6. Functional Interface

- An interface with exactly one abstract method is called Functional Interface.
- It can be used with lambda expressions (@FunctionalInterface).
- It promotes functional programming in Java.

Functional Interface → Used with Lambda Expression → `() -> { code }`

### 7. Built-in Functional Interfaces (Most Used)

| Interface | Package | Method | Description | Example |
|---|---|---|---|---|
| Predicate<T> | java.util.function | boolean test(T t) | Takes input, returns boolean | t -> t.length() > 5 |
| Function<T,R> | java.util.function | R apply(T t) | Takes input, returns result | s -> s.toUpperCase() |
| Consumer<T> | java.util.function | void accept(T t) | Takes input, returns nothing | s -> System.out.println(s) |
| Supplier<T> | java.util.function | T get() | Takes nothing, returns result | () -> new ArrayList<() |

### 8. Functional Interface Example

```
@FunctionalInterface
interface MyFunctional {
    void show(String msg);   // only one abstract method
}
public class Test {
    public static void main(String[] args) {
        MyFunctional mf = (msg) -> System.out.println(msg);
        mf.show("Hello Java 8");
    }
}
```

Lambda expression using functional interface.

### 9. Summary

- ☑ Interfaces in Java 8 became more powerful.
- ☑ Default, Static and Private methods improved reusability.
- ☑ Functional Interfaces enable lambda expressions.
- ☑ Java 8 interfaces are the backbone of modern Java.

**Remember** : Functional Interface + Lambda = Powerful Java 8

@aman_views

# Java Abstraction and Interface

## 7. LAMBDA EXPRESSIONS - COMPLETE GUIDE

### 1. What is Lambda Expression ?

Lambda Expression is a short way to implement a Functional Interface.
It helps in writing clean, concise and readable code.

**Syntax :**

```
(parameters) → expression
        or
(parameters) → { statements; }
```

### 2. Why Lambda is Possible ?

- Functional Interface has only one abstract method.
- JVM knows exactly which method to implement.
- So we can provide implementation using → .

Before Java 8 (using Anonymous Class)

```
Runnable r = new Runnable() {
    public void run() {
        System.out.println("Hello");
    }
};
```

→

After Java 8 (Lambda)

```
Runnable r = () →
    System.out.println("Hello");
```

### 3. Lambda Examples

| No Parameter | One Parameter | Multiple Parameters | With Multiple Statements |
|---|---|---|---|
| `Runnable r = () → System.out.println("Hi");` | `Consumer<String> c = (name) → System.out.println(name);` | `BiFunction<Integer, Integer, Integer> add = (a, b) → a + b;` | `Comparator<Integer> comp = (a, b) → { if(a > b) return 1; else if(a < b) return -1; else return 0; };` |

### 4. Lambda with Functional Interface

| Functional Interface | Method | Lambda Example | Use Case |
|---|---|---|---|
| Predicate<T> | boolean test(T t) | x → x % 2 == 0 | Check condition |
| Function<T,R> | R apply(T t) | s → s.length() | Transform input to output |
| Consumer<T> | void accept(T t) | x → System.out.println(x) | Consume input |
| Supplier<T> | T get() | () → new ArrayList<>() | Provide / Supply data |

### 5. Method Reference (::)

- Another way to write lambda.

| Lambda | Method Reference |
|---|---|
| s → s.toUpperCase() | String::toUpperCase |
| System.out::println | System.out::println |
| () → Math.random() | Math::random |
| list → list.size() | List::size |

### 6. Before vs After Java 8

Before Java 8 (Verbose Code)

```
List<String> list = Arrays.asList("A","B","C");
List<String> upper = new ArrayList<>();
for(String s : list) {
    upper.add(s.toUpperCase());
}
System.out.println(upper);
```

→

After Java 8 (Lambda + Stream)

```
List<String> list = Arrays.asList("A","B","C");

List<String> upper = list.stream()
        .map(String::toUpperCase)
        .collect(Collectors.toList());
System.out.println(upper);
```

### 7. Where Lambda is Used ?

- ☑ Collection Framework (forEach, sort, filter)
- ☑ Stream API (map, filter, collect)
- ☑ Event Handling (GUI, Web)
- ☑ Multithreading (Runnable, Callable)
- ☑ Functional Programming in Java

### 8. Summary

- Lambda makes code short, clean and easy to read.
- Works only with Functional Interface.
- Reduces boilerplate code (Anonymous class).
- Enables Functional Programming in Java.
- Widely used in real world (Streams, Collections, APIs).

**Remember** : Functional Interface + Lambda = Modern Java (Clean, Concise, Powerful)

# OOPS CONCEPT FULL HANDBOOK

## 9. SUMMARISE ALL 4 OOPS PILLARS

OOPs is built on four main pillars. Together, they help us design robust, reusable and maintainable software.

| Pillar | Meaning | Key Points | Example (in Java) |
|---|---|---|---|
| **1. Encapsulation**<br>Wrapping data and methods together as a single unit and restricting direct access. | It protects the data and hides internal details using access modifiers. | • Achieved using private variables.<br>• Access provided through public getter and setter methods.<br>• Improves security and maintainability. | `class Student {`<br>`  private int id;`<br>`  private String name;`<br>`  public int getId() {`<br>`  return id;`<br>`  }`<br>`}` |
| **2. Abstraction**<br>Hiding implementation details and showing only essential features. | It focuses on what an object does instead of how it does it. | • Achieved using abstract classes and interfaces.<br>• Reduces complexity.<br>• Helps in defining contracts. | `abstract class Shape {`<br>`  abstract void draw();`<br>`}`<br>`class Circle extends Shape {`<br>`  void draw() {`<br>`  // implementation`<br>`  }`<br>`}` |
| **3. Inheritance**<br>Acquiring properties and behaviors of one class into another class. | It promotes code reusability and establishes an IS-A relationship. | • Achieved using extends (class) and implements (interface).<br>• Child class inherits fields and methods of parent class.<br>• Supports method overriding. | `class Animal {`<br>`  void eat() { }`<br>`}`<br>`class Dog extends Animal {`<br>`  void bark() { }`<br>`}` |
| **4. Polymorphism**<br>Ability of an object to take many forms. | It allows one interface to be used for different types. | • Achieved through method overloading and method overriding.<br>• Increases flexibility.<br>• Supports compile-time and run-time polymorphism. | `class Animal {`<br>`  void sound() { }`<br>`}`<br>`class Dog extends Animal {`<br>`  @Override`<br>`  void sound() {`<br>`  System.out.println("Bark");`<br>`  }`<br>`}` |

### In Short

- ✓ Encapsulation protects data and restricts access.
- ✓ Abstraction hides details and shows only essential features.
- ✓ Inheritance allows code reusability through IS-A relationship.
- ✓ Polymorphism allows one interface to work with multiple forms.

### Together, these four pillars:

- Make code reusable
- Reduce complexity
- Improve flexibility
- Increase security and maintainability